

Apply filters to SQL queries

Project description

My organization is working to strengthen the security of its systems. As part of this work, I used SQL to investigate potential security issues, analyze login activity, and identify employee devices that required security updates. The following tasks demonstrate how I used SQL filters to extract relevant information from organizational data.

Retrieve after hours failed login attempts

A potential security incident occurred after business hours, defined as after 18:00. I needed to identify unsuccessful login attempts during this period so they could be reviewed for suspicious activity.

The following code demonstrates how I created a SQL query to filter for failed login attempts that occurred after business hours.

```
MariaDB [organization]> SELECT *  
  -> FROM log_in_attempts  
  -> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0

The first part of the screenshot shows my query, while the second part displays a portion of the results. The query uses the `log_in_attempts` table to isolate unsuccessful login attempts made after 18:00. I used a `WHERE` clause with the `AND` operator to apply both conditions at the same time. The `login_time > '18:00'` condition identifies attempts made after business hours, while `success = FALSE` limits the results to unsuccessful attempts.

Retrieve login attempts on specific dates

A suspicious event occurred on 2022-05-09, so login activity from that date and the previous day needed to be examined.

The following code demonstrates how I created a SQL query to filter for login attempts that occurred on specific dates.

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0

The first part of the screenshot shows my query, and the second part shows a portion of the results. I used a **WHERE** clause with the **OR** operator to retrieve login attempts from either 2022-05-09 or 2022-05-08. The first condition, **login_date = '2022-05-09'**, identifies activity on the date of the incident, while **login_date = '2022-05-08'** includes activity from the preceding day. Using **OR** allowed me to investigate both dates within a single query.

Retrieve login attempts outside of Mexico

After reviewing the organization's login data, I identified activity originating from outside Mexico that required further investigation.

The following code demonstrates how I created a SQL query to filter for login attempts that occurred outside of Mexico.

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0

The first part of the screenshot shows my query, and the second part shows a portion of the results. I used the **NOT** operator with **LIKE** to exclude login attempts associated with Mexico. I used **MEX%** as the pattern because the dataset represents Mexico using both **MEX** and **MEXICO**. The **%** wildcard allows the query to match any additional characters following **MEX**, making it possible to account for both formats in the dataset.

Retrieve employees in Marketing

My team needed to update the computers assigned to employees in the Marketing department. I therefore needed to identify the relevant employees located in the East building.

The following code demonstrates how I created a SQL query to filter for employee machines from employees in the Marketing department in the East building.

```
MariaDB [organization]> SELECT *  
-> FROM employees  
-> WHERE department = 'Marketing' AND office LIKE 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267

The first part of the screenshot shows my query, and the second part contains a portion of the output. I used the `employees` table and combined two conditions with the `AND` operator. The `department = 'Marketing'` condition identifies employees in the Marketing department, while `office LIKE 'East%'` identifies employees working in the East building. The `%` wildcard is useful here because the office field includes specific office numbers after the word `East`.

Retrieve employees in Finance or Sales

Employees in the Finance and Sales departments also required a specific security update. I needed to identify employees belonging to either of these departments so their machines could be included in the update.

The following code demonstrates how I created a SQL query to filter for employee machines from employees in the Finance or Sales departments.

```

MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
|          1003 | d394e816f943 | sgilmore | Finance | South-153 |
|          1007 | h174i497j413 | wjaffrey | Finance | North-406 |
|          1008 | i858j583k571 | abernard | Finance | South-170 |

```

The first part of the screenshot shows my query, and the second part shows a portion of the results. I used the **OR** operator to retrieve employees from either department. The condition **department = 'Finance'** identifies Finance employees, while **department = 'Sales'** identifies Sales employees. Using **OR** ensures that records matching either condition are included in the results.

Retrieve all employees not in IT

My team also needed to apply a security update to employees outside of the Information Technology department. Before carrying out the update, I needed to identify the affected employees and their machines.

The following demonstrates how I created a SQL query to filter for employee machines from employees not in the Information Technology department.

```

MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE NOT department = 'Information Technology';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
|          1000 | a320b137c219 | elarson | Marketing | East-170 |
|          1001 | b239c825d303 | bmoreno | Marketing | Central-276 |
|          1002 | c116d593e558 | tshah | Human Resources | North-434 |

```

The first part of the screenshot shows my query, and the second part displays a portion of the results. I used a **WHERE** clause with the **NOT** operator to exclude employees whose department is Information Technology. This allowed me to focus specifically on employees outside the IT department who needed the security update.

Summary

I used SQL filtering to investigate login activity and identify employee machines relevant to several security tasks. Working with the `log_in_attempts` and `employees` tables, I applied `AND`, `OR`, and `NOT` to combine, expand, or exclude specific conditions. I also used the `LIKE` operator and `%` wildcard to work with patterns in the organization's data. These queries allowed me to narrow large datasets into targeted results that could support security investigations, incident analysis, and system updates.